pyglenn: A Python Toolkit for Thermochemical Properties Calculation from NASA Polynomials

Dr. Reginaldo G. Leão Jr.

August 2026

Abstract

This paper presents pyglenn 0.2.0, an open-source Python package for the computation of thermochemical properties — heat capacity (C_p), enthalpy (H°), and entropy (S°) — from NASA-7 polynomial coefficients. The package parses legacy FORTRAN `thermo.inp` files into a normalized SQLite database, exposes a typed, object-oriented Python API, and ships a command-line interface. The bundled dataset covers 2,035 chemical species and 3,779 temperature intervals and is denormalized with the dataset's original NASA Glenn/CEA reference gas constant ($R = 8.314510$ J/(mol · K), CODATA 1986), which eliminates a systematic ~5.7 ppm bias that arises when the modern CODATA universal value is used instead. Cross-validation against NIST-JANAF reference data demonstrates subpercent agreement for heat capacity and entropy across seven reference species. pyglenn is dependency-free, installable from PyPI, and suitable for researchers and engineers in combustion science, chemical thermodynamics, and related fields.

1. Introduction

Accurate thermochemical data are fundamental to modeling and simulating reactive flows, combustion processes, and chemical equilibrium. The NASA polynomial format, originally developed for the NASA Glenn Research Center's chemical equilibrium program (CEA), has become a de facto standard for representing temperature-dependent thermodynamic properties of chemical species over wide temperature ranges [1, 2].

The `thermo.inp` files distributed with legacy FORTRAN codes contain polynomial coefficients for thousands of species, but their rigid fixed-column FORTRAN layout makes them cumbersome to parse in modern computational workflows. Beyond parsing, a subtle scientific pitfall exists: the NASA Glenn/CEA polynomials are dimensionless quantities (C_p/R, H/RT, S/R) that were fitted and tabulated using the CODATA 1986 gas constant ($R = 8.314510$ J/(mol · K)). Denormalizing them with the modern CODATA 2018 universal

value ($R = 8.31446261815324$ J/(mol · K)) introduces a systematic bias of approximately −5.7 ppm in every dimensional result. **pyglenn** addresses both problems by:

- Parsing FORTRAN-formatted `thermo.inp` files into a structured, normalized SQLite3 database using an explicit, state-machine parser that is resilient to malformed input;
- Recording the dataset's reference gas constant in database metadata and denormalizing every result with it, falling back to the universal value only for legacy databases;
- Providing a clean, **typed** Python API whose public methods return frozen dataclasses (not opaque dictionaries), plus a command-line interface;
- Bundling a pre-built, integrity-validated database so that users require no manual setup;
- Offering a reproducible database-rebuild and validation toolchain, backed by a cross-validation audit against NIST-JANAF reference data.

The package is distributed under the MIT license and is available via PyPI (`pip install pyglenn`); a conda recipe is provided for conda-based installation.

2. Theoretical Background

2.1 NASA-7 Polynomial Format

The NASA-7 representation expresses dimensionless thermodynamic quantities as functions of temperature T [K] using seven coefficients $(a_1, a_2, \dots, a_7)$ plus two integration constants (b_1, b_2) per temperature interval. The exact forms implemented by pyglenn are:

$$\frac{C_p^\circ}{R} = \frac{a_1}{T^2} + \frac{a_2}{T} + a_3 + a_4 T + a_5 T^2 + a_6 T^3 + a_7 T^4 \tag{1}$$

$$\frac{H^\circ}{RT} = -\frac{a_1}{T^2} + a_2 \frac{\ln T}{T} + a_3 + \frac{a_4}{2} T + \frac{a_5}{3} T^2 + \frac{a_6}{4} T^3 + \frac{a_7}{5} T^4 + \frac{b_1}{T} \tag{2}$$

$$\frac{S^\circ}{R} = -\frac{a_1}{2T^2} - \frac{a_2}{T} + a_3 \ln T + a_4 T + \frac{a_5}{2} T^2 + \frac{a_6}{3} T^3 + \frac{a_7}{4} T^4 + b_2 \tag{3}$$

The integration constant b_1 encodes the enthalpy reference (the enthalpy of formation at 298.15 K), and b_2 encodes the entropy reference; consequently $H^\circ(298.15\,\mathrm{K})$ equals the standard enthalpy of formation $\Delta H_f^\circ(298.15)$. Each species is split into one or more temperature intervals (typically two, but up to three or more for species with condensed phases), and each interval carries its own coefficient set.

Dimensional properties are recovered by denormalization with the dataset's reference gas constant R_{ref}:

$$C_p = \frac{C_p^\circ}{R} R_{\text{ref}}, \qquad H^\circ = \frac{H^\circ}{RT} R_{\text{ref}} T, \qquad S^\circ = \frac{S^\circ}{R} R_{\text{ref}} \tag{4}$$

pyglenn distinguishes two constants explicitly (Table 1). The bundled NASA Glenn/CEA data were fitted with the CODATA 1986 value, which is persisted in the database `metadata` table and selected automatically at connection time.

Symbol	Value (J/(mol · K))	Meaning
`R_UNIVERSAL`	8.31446261815324	CODATA 2018/2022 universal value
`R_GLENN`	8.314510	NASA Glenn/CEA `thermo.inp` reference (CODATA 1986)

Table 1. Gas constants exposed by pyglenn.

2.2 The `thermo.inp` FORTRAN Format

The `thermo.inp` file follows a rigid fixed-column structure defined in the NASA Glenn report [1]. Each species occupies five record types:

Record	Content
1	Species name and comments (A16, A62)
2	Number of intervals, reference code, phase, molecular weight, $\Delta H_f^\circ(298.15)$
3	Temperature range (T_{min}, T_{max}, T_{mid})
4	First five coefficients a_1–a_5 (5D16.9)
5	Remaining coefficients a_6, a_7 and integration constants b_1, b_2

Records 3–5 repeat for each temperature interval. Two properties make the format error-prone in practice: the fixed-column layout (a single misplaced character shifts every field), and FORTRAN's use of `D` instead of `E` in scientific notation (e.g. `1.234567890D+05`). pyglenn addresses these with dedicated record-type detectors (temperature-line and coefficient-line classifiers) and a strict, state-machine parser that validates each species block before insertion, so that a malformed block can never be silently recovered as a new species.

3. Architecture and Data Model

pyglenn follows a layered design with three core modules:

- **`builder.py`** — Parses `thermo.inp` into a normalized SQLite database within a single transaction (`BEGIN/COMMIT/ROLLBACK`). The parser consumes only the record type expected at each state (`species header → general information → N complete interval triplets`), rejects malformed blocks, and resynchronizes only at a verified species-header boundary.
- **`database.py`** — Provides low-level SQL querying, species lookup, the numerical evaluation of Equations (1)–(3), dataset-metadata handling (reference gas constant), and a reusable integrity validator.
- **`calculator.py`** — High-level interface exposing scalar, range, and enthalpy-change calculations with typed return values and rich, context-carrying exceptions.

The normalized schema comprises five tables:

Table	Purpose
`species`	Species identity, phase, molecular weight, formation enthalpy
`temperature_intervals`	Per-species temperature bounds and reference enthalpy
`coefficients`	The nine NASA-7 values (a_1–a_7, b_1, b_2) per interval
`file_metadata`	Global `thermo.inp` metadata (global temperature bounds, date)
`metadata`	Additive key/value metadata, including the dataset gas constant

The public API returns **frozen dataclasses** (`ThermoProperties`, `SpeciesInfo`, `NASACoefficients`, `IntervalData`, `SpeciesData`, and `DatabaseStats`) instead of untyped dictionaries. This improves static checking, IDE autocompletion, and self-documentation, while `dataclasses.asdict()` remains available for JSON serialization and legacy callers.

The bundled database contains **2,035 species** (1,266 gaseous and 769 condensed) spanning cryogenic to multi-thousand-Kelvin ranges, with **3,779 temperature intervals** and **3,779 coefficient sets**. The average molecular weight of the curated dataset is approximately 132 g/mol.

4. Python API

The primary interface is the `ThermochemicalCalculator` class, which supports the context-manager protocol for automatic resource management:

```python
from pyglenn import ThermochemicalCalculator

with ThermochemicalCalculator() as calc:
    species = calc.get_available_species("CO2", exact_match=True)
    props = calc.calculate_properties(species[0].id, 1000.0)
    print(f"Cp = {props.cp:.2f} J/(mol·K)")
    print(f"H° = {props.h_relative:.1f} J/mol")
    print(f"S° = {props.s:.3f} J/(mol·K)")
```

No explicit database path is required — the package ships with a pre-built, integrity-validated `thermo.db`. Users may supply a custom database file, and legacy databases without the `metadata` table are migrated automatically on connection (the additive metadata table is created; thermochemical rows are never modified).

Key methods include:

- `calculate_properties(species_id, T) -> ThermoProperties` — scalar evaluation at a single temperature;
- `calculate_properties_range(species_id, temps, strict=False) -> list[ThermoProperties]` — bulk evaluation that loads the species intervals once and evaluates all temperatures in memory (preserving input order);
- `get_properties_range(species_id, temps) -> dict[float, ThermoProperties]` — temperature-keyed dictionary for backward compatibility;
- `calculate_formation_enthalpy(species_id)` and `calculate_enthalpy_change(species_id, T1, T2)`;
- `get_available_species(pattern, exact_match=False)`, `list_all_species()`, `get_species_info()`, `get_species_data()`, and `get_statistics()` for database introspection;
- `validate_database()` — integrity checks (foreign keys, one coefficient set per interval, no orphan rows, ordered bounds, `num_intervals` agreement, semantic guards), used by the rebuild toolchain and the CLI.

Errors are reported through a formal exception hierarchy: `ThermoCalcError` (base), `DatabaseNotConnectedError`, `SpeciesNotFoundError` (carrying the requested `species_id`), and `TemperatureOutOfRangeError` (carrying the requested temperature, species name, and the valid interval bounds).

5. Command-Line Interface

pyglenn ships a CLI for rapid inspection and maintenance:

```
# Build or rebuild the database from thermo.inp
pyglenn build -i thermo.inp -o thermo.db
```

```
# Query a species
pyglenn query -s O2

# Ensure an existing database carries the canonical dataset metadata
pyglenn migrate-metadata -d thermo.db

# Run read-only integrity checks
pyglenn validate -d thermo.db
```

The `build` subcommand is needed only when users modify the source `thermo.inp` or wish to regenerate a database. A reproducible rebuild script (`scripts/rebuild_database.py`) rebuilds into a temporary file, validates it, and atomically replaces the bundled asset only after every release gate passes.

6. Validation

6.1 NIST-JANAF Cross-Validation

The calculated properties were cross-validated against NIST-JANAF reference tables (Chase, 1998), evaluated via the Shomate equation [3], for seven species over 300–3000 K (Table 2). Heat capacity and entropy agree within a fraction of a percent across all species; the enthalpy column reflects relative errors on sensible enthalpy, which diverge near 298.15 K as the reference value approaches zero.

Species	Mean Cp error (%)	Mean S° error (%)
CO2	−0.025	−0.002
N2	−0.001	+0.000
CO	+0.025	+0.002
H2O	+0.845	+0.053
O2	+0.101	+0.008
NH3	−0.631	−0.117
SO2	−0.463	−0.067

Table 2. Mean relative error of pyglenn vs. NIST-JANAF (Chase, 1998).

6.2 Database Integrity

`validate_database()` and the `pyglenn validate` command verify foreign keys, the one-to-one interval–coefficient relationship, the absence of orphan rows, finite and ordered temperature bounds, agreement between the declared and actual interval counts, and semantic guards (no coefficient-like species names, finite positive molecular weights). The bundled database passes all checks, and the rebuild script runs the same validation before replacing the asset.

7. Conclusion

pyglenn fills a practical niche in the computational thermodynamics ecosystem: it transforms the widely used but format-locked NASA polynomial data into an accessible, scriptable, and dependency-free Python tool. Version 0.2.0 adds a typed, dataclass-based public API; a correct, dataset-specific gas-constant handling (eliminating a ~5.7 ppm systematic bias); a robust state-machine parser; automatic metadata migration; database-integrity validation; and a NIST-JANAF cross-validation audit.

The package is lightweight (zero runtime dependencies beyond the Python standard library), installable from PyPI, supports Python 3.9–3.12, and provides both an intuitive Python API and a command-line interface. Future work may include support for the NASA-9 polynomial format, integration with chemical-equilibrium solvers, and a web-based interactive species browser.

References

[1] B. J. McBride, M. J. Zehe, and S. Gordon, "NASA Glenn Coefficients for Calculating Thermodynamic Properties of Individual Species," NASA/TP—2002-211556, NASA Glenn Research Center, Cleveland, OH, September 2002.

[2] S. Gordon and B. J. McBride, "Computer Program for Calculation of Complex Chemical Equilibrium Compositions and Applications," NASA RP-1311, NASA Lewis Research Center, Cleveland, OH, October 1994.

[3] M. W. Chase, Jr., "NIST-JANAF Thermochemical Tables," 4th ed., J. Phys. Chem. Ref. Data, Monograph 9, 1998.